Architecture Decision Records (ADR): RAG-Diary

> The significant architecture decisions, with their context, the alternatives considered and the consequences, so that whoever comes later understands why the system is the way it is.

> Thai version: ADR.md

1. Document control

Item	Detail
Document	Architecture Decision Records
Version	1.0
Prepared	15 October 2026
Author	Business Analyst / System Analyst (project owner)
Status	Reviewed, round 1 (25 October 2026)
Related documents	C4-ARCHITECTURE, DATA-MODEL, DFD-PRIVACY

2. Purpose

This document answers one question for each decision: what was decided, why, and what was traded away? It follows Michael Nygard's format: status, context, decision, alternatives considered, and consequences.

3. Decision index

ID	Topic	Status
ADR-01	Split the frontend and the backend into separate parts	Accepted
ADR-02	Use PostgreSQL with pgvector instead of a dedicated vector database	Accepted
ADR-03	Enforce data isolation with row-level security	Accepted
ADR-04	Use OpenAI as the AI provider	Superseded by ADR-08

ID	Topic	Status
ADR-05	Use retrieval-augmented generation for question answering	Accepted
ADR-06	Keep the session token on the client	Superseded by ADR-10
ADR-07	Use a modular monolith structure	Accepted
ADR-08	Move the AI provider to Google Gemini (free tier)	Accepted
ADR-09	Set the search performance target to p95 of 3.5 seconds	Accepted
ADR-10	Keep the session in httpOnly cookies behind a same-origin proxy	Accepted

4. ADR-01 Split the frontend and the backend into separate parts

Status: Accepted

Context: The system needs a responsive screen and business logic that talks to the database and the AI provider. Putting everything in one place tends to become hard to maintain as it grows.

Decision: Split into two clear parts. The frontend uses Next.js and the backend uses Elysia, and they talk over an API.

Alternatives: (a) a single full-stack app with the server inside Next.js; (b) two separate parts, as chosen.

Consequences: Each part can be tested and deployed on its own. The cost is that the API contract must be kept in step and CORS has to be configured.

5. ADR-02 Use PostgreSQL with pgvector

Status: Accepted

Context: Entries and a vector index have to be stored to support retrieval by meaning.

Decision: Use PostgreSQL with the pgvector extension and an HNSW index in the same database, rather than a dedicated vector database.

Alternatives: (a) a dedicated vector database such as Pinecone or Qdrant; (b) pgvector inside PostgreSQL, as chosen.

Consequences: One source of data, less complexity, and RLS applies to vectors too. At very large scale, a dedicated vector database may perform better.

6. ADR-03 Enforce data isolation with row-level security

Status: Accepted

Context: Diary content is private. It must be impossible to read across users, even if the application layer has a bug.

Decision: Enable RLS and define per-action policies in the database, keyed on auth.uid() matching user_id.

Alternatives: (a) authorise only at the API layer; (b) authorise at both the API and the database, as chosen.

Consequences: Two layers of protection. The cost is that anything running with server privileges must be handled carefully so it does not bypass RLS by accident.

7. ADR-04 Use OpenAI as the AI provider

Status: Superseded (replaced by ADR-08)

Context: A provider was needed for embeddings, answers and images.

Decision: Use OpenAI for all three: 1,536-dimension embeddings, a chat model and an image model.

Alternatives: (a) mix several providers; (b) a single provider, as chosen.

Consequences: One key and one bill to manage. The cost is dependence on a single provider and on that provider's availability.

8. ADR-05 Use retrieval-augmented generation for question answering

Status: Accepted

Context: Users want to ask questions about their own entries, not about general knowledge.

Decision: Retrieve first, then answer. Find the user's related entries and feed them to the model as context.

Alternatives: (a) let the model answer from memory without retrieval; (b) use RAG, as chosen.

Consequences: Answers are grounded in the user's real data, which reduces invented content. The quality depends on the index and on retrieval.

9. ADR-06 Keep the session token on the client

Status: Superseded by ADR-10

Context: The authentication service issues stateless tokens, so a storage location had to be chosen.

Decision: Keep the token on the client in localStorage and attach it to every request in the Authorization header.

Alternatives: (a) store it in an httpOnly cookie that scripts cannot read; (b) localStorage, as chosen.

Consequences: Simple for the frontend and fast to develop against. The risk is theft through a script, which is why moving to an httpOnly cookie was flagged for the next round.

10. ADR-07 Use a modular monolith structure

Status: Accepted

Context: The system has three main domains: authentication, entry management and AI processing.

Decision: One deployable service, split into modules by domain: auth, notes and ai.

Alternatives: (a) several microservices; (b) a single set of modules, as chosen.

Consequences: Easy to deploy and a good fit for a small team. Per-module scaling is limited if the system grows much larger.

11. ADR-08 Move the AI provider to Google Gemini (free tier)

Status: Accepted

Context: The previous provider was OpenAI, but the account credit ran out, so embeddings, answers and images all stopped. The system needed a provider that worked without adding

cost first.

Decision: Use Google Gemini through a single key from Google AI Studio, split as follows:

- Embeddings: `gemini-embedding-001` with `outputDimensionality=1536` to match the existing `vector(1536)` column, so the schema did not change.
- Chat and mood summaries: `gemini-3.5-flash` through the OpenAI-compatible endpoint.
- Images: `gemini-3.1-flash-image`, subject to the plan quota; the free tier may answer 429, and the system then reports that the feature is unavailable.

Alternatives: (a) top up OpenAI credit; (b) run embeddings locally through Ollama with opencode-go for chat; (c) Gemini, as chosen.

Consequences: One key, a free tier, and embeddings that match the existing schema. The costs are a limited image quota, model names that change often (gemini-2.5-flash was withdrawn for new users), and dependence on one provider, which is why model names are read from the environment so they can be swapped without code changes.

12. ADR-09 Set the search performance target to p95 of 3.5 seconds

Status: Accepted

Context: NFR-01 originally required p95 of 2 seconds. That number came from practice, not from measurement. Measured on production with 30 entries and 20 queries, the result was p50 2.39 seconds and p95 3.16 seconds, with most of the time spent in the external embedding call that sits on every query.

Decision: Change NFR-01 to p95 of 3.5 seconds, counting the embedding call, and keep a warning threshold at 5 seconds.

Alternatives: (a) keep 2 seconds and change the embedding provider; (b) cache query embeddings; (c) align the target with real use, as chosen.

Consequences: The target matches reality and current cost, and users still get an acceptable response for diary search. The downside is that it removes the pressure to change the architecture now, so p95 has to be watched and the target revisited if it exceeds 3.5 seconds.

13. ADR-10 Keep the session in httpOnly cookies behind a same-origin proxy

Status: Accepted

Context: ADR-06 kept the token in localStorage, where any script can read it. An XSS flaw would let an attacker take the token and act as the user. The token had to move somewhere JavaScript cannot read.

Decision: Keep both tokens in httpOnly cookies with SameSite=Lax and Path=/, and Secure in production. The browser talks only to the Next.js origin, and Next.js rewrites /api/* to the Elysia server, which makes the cookie first-party and avoids SameSite=None, which modern browsers block. The server still accepts an Authorization header as a fallback for scripts, and checks the Origin of non-GET requests as an extra CSRF guard.

Alternatives: (a) keep localStorage; (b) use SameSite=None cookies across domains directly; (c) use a same-origin proxy, as chosen.

Consequences: JavaScript cannot read the token, the impact of XSS is reduced, and no third-party cookie is needed. The cost is an extra proxy layer, slightly harder debugging, and the rule that every request goes through Next.js, so the frontend must not point NEXT_PUBLIC_API_URL at another domain.

14. References

Document	Relationship
C4-ARCHITECTURE	The architecture these decisions produce
DATA-MODEL	The origin of ADR-02 and ADR-03
DFD-PRIVACY	The privacy reasoning that led to ADR-06